TASK 3 – INTERACTIVE TO-DO LIST APPLICATION

JavaScript DOM Manipulation, Event Listeners and Dynamic UI

Task Name:	Interactive To-Do List Application	**File Name:**	`index.html`
Repository:	full-stack-web-development	**GitHub Folder:**	01-Observation/obsTask3_todo_list

1. AIM

To design and implement an interactive To-Do List web application using HTML, CSS, and vanilla JavaScript that allows users to add, edit, mark complete, and delete tasks dynamically with real-time UI updates.

2. TASK STATEMENT

Develop an application using JavaScript, DOM manipulation, and event listeners. The application provides an input text box for entering a task and an "Add Task" button. When clicked, it dynamically creates a new task item and adds it to the list. Each task item contains "Complete", "Edit", and "Delete" buttons to visually toggle completion status, update text content via prompts, or remove the task entirely. An appropriate message is displayed whenever the task list is empty.

3. CONCEPTS DEMONSTRATED

- Selecting DOM elements using `document.getElementById()`
- Creating HTML elements dynamically using `document.createElement()`
- Modifying element properties, text, and styles dynamically
- Attaching click event listeners using `addEventListener()`
- Dynamic addition and removal of DOM elements via `appendChild()` and `remove()`
- Conditional rendering to show or hide the empty state message
- Interactive web UI development without page reloads

4. PROGRAM WORKING

- When page loads, if no tasks exist, a "No tasks available." message is rendered.
- The user types a task name in the input field and clicks "Add Task".
- A new list item is dynamically generated along with three action buttons (Complete, Edit, Delete).
- Clicking "Complete" toggles line-through styling on the task text.
- Clicking "Edit" opens a prompt modal allowing the user to modify the task description.
- Clicking "Delete" removes the corresponding task element from the DOM and restores the empty message if list count becomes zero.

5. COMPLETE SOURCE CODE

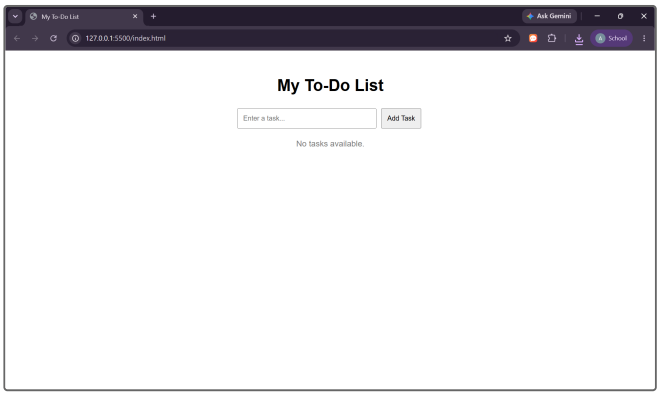
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My To-Do List</title>
    <style>
        body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; background-color: #ffffff; }
        h1 { font-size: 2rem; font-weight: bold; margin-bottom: 20px; }
        .input-container { margin-bottom: 15px; }
        input[type="text"] { width: 250px; padding: 8px; font-size: 14px; border: 1px solid #ccc; border-radius:
2px; }
        button { padding: 8px 12px; font-size: 14px; cursor: pointer; border: 1px solid #777; background: #f0f0f0;
border-radius: 2px; margin-left: 2px; }
        #taskList { list-style-type: none; padding: 0; max-width: 450px; margin: 0 auto; }
        .task-item { display: flex; align-items: center; justify-content: space-between; margin-bottom: 10px; font-
size: 15px; }
        .completed { text-decoration: line-through; color: #777; }
        .empty-msg { color: #777; font-size: 15px; margin-top: 15px; }
    </style>
</head>
<body>
    <h1>My To-Do List</h1>
    <div class="input-container">
        <input type="text" id="taskInput" placeholder="Enter a task...">
        <button id="addBtn">Add Task</button>
    </div>
    <ul id="taskList"></ul>
    <div id="emptyMsg" class="empty-msg">No tasks available.</div>

    <script>
        const taskInput = document.getElementById("taskInput");
        const addBtn = document.getElementById("addBtn");
        const taskList = document.getElementById("taskList");
```

```
        const emptyMsg = document.getElementById("emptyMsg");

        function updateEmptyState() {
            emptyMsg.style.display = taskList.children.length === 0 ? "block" : "none";
        }

        addBtn.addEventListener("click", function() {
            const text = taskInput.value.trim();
            if (text === "") return;

            const li = document.createElement("li");
            li.className = "task-item";

            const span = document.createElement("span");
            span.textContent = text;

            const btnContainer = document.createElement("div");

            const completeBtn = document.createElement("button");
            completeBtn.textContent = "Complete";
            completeBtn.addEventListener("click", () => span.classList.toggle("completed"));

            const editBtn = document.createElement("button");
            editBtn.textContent = "Edit";
            editBtn.addEventListener("click", () => {
                const newText = prompt("Edit your task:", span.textContent);
                if (newText !== null && newText.trim() !== "") span.textContent = newText.trim();
            });

            const deleteBtn = document.createElement("button");
            deleteBtn.textContent = "Delete";
            deleteBtn.addEventListener("click", () => {
                li.remove();
                updateEmptyState();
            });

            btnContainer.appendChild(completeBtn);
            btnContainer.appendChild(editBtn);
            btnContainer.appendChild(deleteBtn);

            li.appendChild(span);
            li.appendChild(btnContainer);
            taskList.appendChild(li);

            taskInput.value = "";
            updateEmptyState();
        });
        updateEmptyState();
    </script>
</body>
</html>
```

6. EXPECTED OUTPUT & EXECUTION SCREENSHOTS

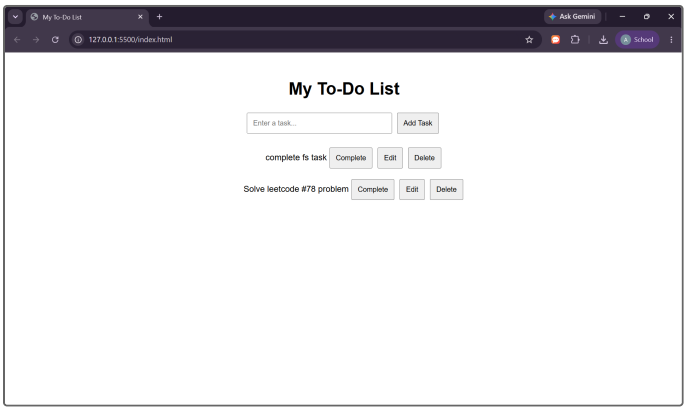
Screenshot 1: Initial Empty State

Displays "No tasks available." message when no task items exist in list.



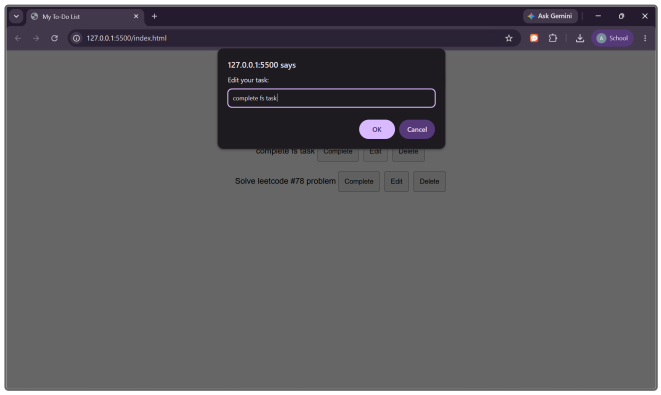
Screenshot 2: Dynamically Adding Tasks

Tasks added with "Complete", "Edit", and "Delete" buttons.



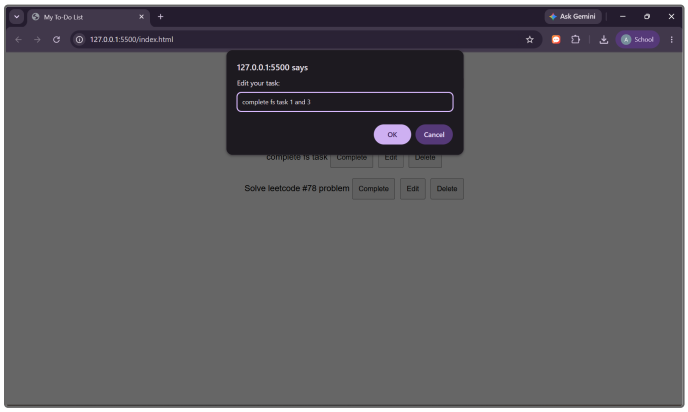
Screenshot 3: Triggering Edit Task Modal

Clicking "Edit" triggers a prompt window with current task text.



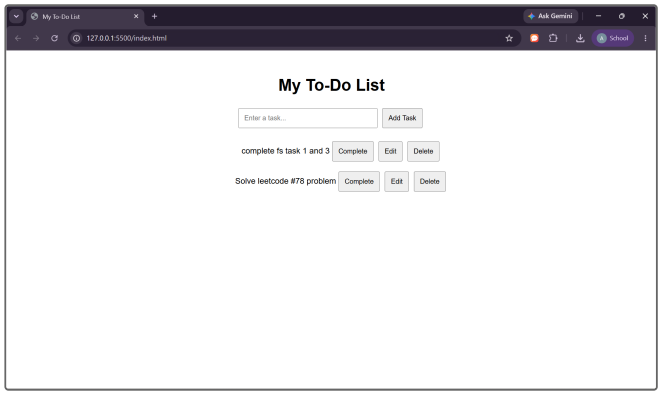
Screenshot 4: Modifying Task Text

Updating input text in prompt to "complete fs task 1 and 3".



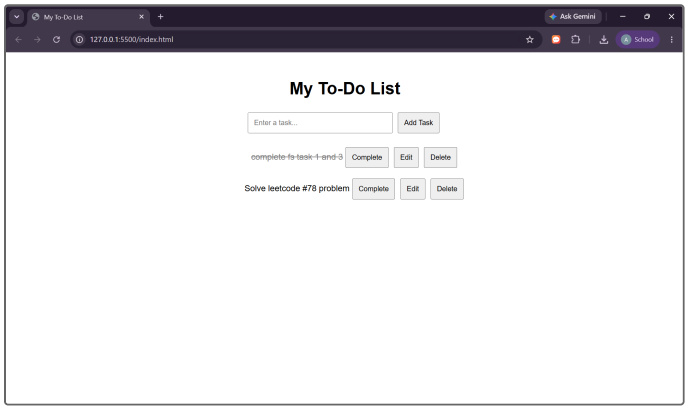
Screenshot 5: Task Updated in DOM

DOM reflects modified text instantly without reloading page.



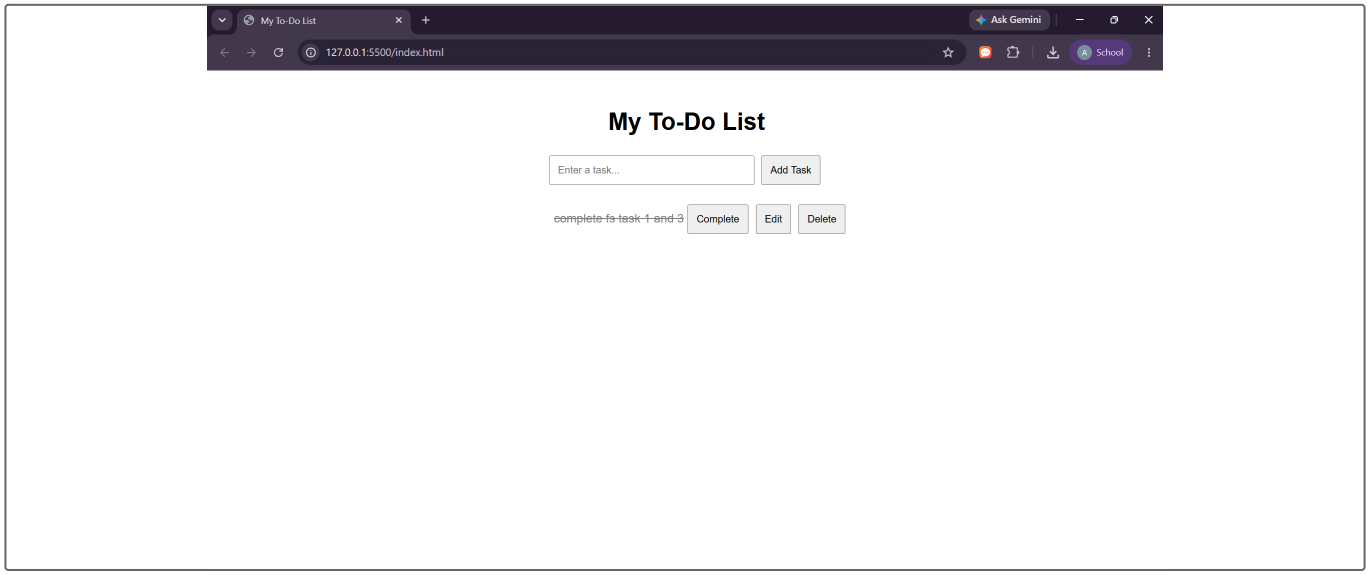
Screenshot 6: Marking Task Complete

Clicking "Complete" applies line-through strike-through formatting.



Screenshot 7: Task Deletion & Final State

Clicking "Delete" removes task item dynamically from DOM tree while remaining tasks stay unaffected.



7. KEY CODE EXPLANATION

- `document.getElementById()` — Grabs key DOM elements like input, list, button, and empty state container.
- `document.createElement("li")` — Dynamically creates list item elements when user submits new tasks.
- `addEventListener("click", ...)` — Attaches event handlers for task creation, text editing, status toggling, and item deletion.
- `span.classList.toggle("completed")` — Toggles CSS strikethrough styling to visually indicate task completion.
- `li.remove()` — Removes selected task node from document object model directly.
- `updateEmptyState()` — Checks list length and dynamically shows/hides "No tasks available." text.

8. LEARNING OUTCOMES

- Mastered DOM selection and dynamic HTML node creation using vanilla JavaScript.
- Implemented interactive event handling for dynamic list element manipulation.
- Applied inline CSS class toggling for status visualization.
- Managed UI state dynamically based on user interactions.

9. GITHUB SUBMISSION

The source code for this project is hosted on GitHub:
`https://github.com/venugopalreddychittela/full-stack-web-development/tree/main/01-Observation/obsTask3_todo_list`